

- [Observability Dashboard - Complete Documentation](#)
 - [Table of Contents](#)
 - [Global Constants](#)
 - [1. Total Requests Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)
 - [2. Input Tokens Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)
 - [3. Output Tokens Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)
 - [4. Total Tokens Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)
 - [5. Avg Latency Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)
 - [6. Input Cost Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source - Pricing Strategies](#)
 - [Strategy 1: LiteLLM \(Primary\)](#)
 - [Strategy 2: Internet Search \(Fallback\)](#)
 - [7. Output Cost Card](#)
 - [Definition](#)

- [Formula](#)
- [Code Snippet](#)
- [Data Source](#)
- [8. Total Cost Card](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [How Cost is Calculated Per Request](#)
 - [Data Source](#)
- [Additional Dashboard Sections](#)
- [9. Overview Tab - System Health](#)
 - [Definition](#)
 - [Formulas](#)
 - [Code Snippet](#)
 - [Data Source](#)
- [10. By Model Tab](#)
 - [Definition](#)
 - [Formulas](#)
 - [Code Snippet](#)
 - [Data Source](#)
- [11. Hourly Breakdown Tab](#)
 - [Definition](#)
 - [Formulas](#)
 - [Code Snippet](#)
 - [Data Source](#)
- [12. Recent Requests Tab](#)
 - [Definition](#)
 - [Formula](#)
 - [Code Snippet](#)
 - [Data Source](#)

Observability Dashboard - Complete Documentation

This document explains how each card and section in the Observability Dashboard calculates and displays its data.

Table of Contents

1. [Global Constants](#)
 2. [Total Requests Card](#)
 3. [Input Tokens Card](#)
 4. [Output Tokens Card](#)
 5. [Total Tokens Card](#)
 6. [Avg Latency Card](#)
 7. [Input Cost Card](#)
 8. [Output Cost Card](#)
 9. [Total Cost Card](#)
 10. [Overview Tab - System Health](#)
 11. [By Model Tab](#)
 12. [Hourly Breakdown Tab](#)
 13. [Recent Requests Tab](#)
-

Global Constants

Location: `backend/Agents/observability.py` (Lines 15-32)

These constants are used throughout the observability tracking system:

```
# Token calculation constant
# LLM pricing is quoted per 1 million tokens (e.g., "$5.00 per 1M tokens")
TOKENS_PER_MILLION = 1_000_000

# Web scraping configuration
# Timeout in seconds for HTTP requests when fetching pricing from the internet
WEB_REQUEST_TIMEOUT_SECONDS = 5

# Default limits for data retrieval
# Default number of hours to include in hourly breakdown statistics
DEFAULT_HOURLY_BREAKDOWN_HOURS = 24

# Default number of recent requests to return when querying request history
DEFAULT_RECENT_REQUESTS_LIMIT = 50
```

Variable Explanations:

- **TOKENS_PER_MILLION**: Used to convert token counts to millions for cost calculations (1,000,000 tokens)
 - **WEB_REQUEST_TIMEOUT_SECONDS**: Maximum time to wait for web pricing data before timing out
 - **DEFAULT_HOURLY_BREAKDOWN_HOURS**: How many hours of historical data to display
 - **DEFAULT_RECENT_REQUESTS_LIMIT**: Maximum number of recent requests to return
-

1. Total Requests Card

Definition

Displays the total number of LLM API requests made, along with successful and failed request counts.

Formula

```
Total Requests = count(all requests in self._requests list)
Successful Requests = count(requests where success == True)
Failed Requests = count(requests where success == False)
```

Code Snippet

Location: `backend/Agents/observability.py` (Lines 293-307)

```
# Filter requests by success status
successful = [r for r in self._requests if r.success]
failed = [r for r in self._requests if not r.success]

return {
    "total_requests": len(self._requests),
    "successful_requests": len(successful),
    "failed_requests": len(failed),
    "success_rate": (len(successful) / len(self._requests)) * 100,
```

```
# ...  
}
```

Variable Explanations:

- `self._requests`: List storing all tracked LLM requests (LLMRequest objects)
- `successful`: Filtered list containing only requests where `success=True`
- `failed`: Filtered list containing only requests where `success=False`
- `len(self._requests)`: Count of all requests in the list
- `len(successful)`: Count of successful requests
- `len(failed)`: Count of failed requests

Data Source

Each request is added to `self._requests` when `track_request()` is called after an LLM API call completes.

2. Input Tokens Card

Definition

Displays the total number of input/prompt tokens consumed across all LLM requests.

Formula

```
Total Input Tokens = sum(input_tokens for all requests)
```

Code Snippet

Location: `backend/Agents/observability.py` (Line 296, 308)

```
# Sum all input tokens from all requests  
total_input_tokens = sum(r.input_tokens for r in self._requests)
```

```
return {  
    # ...  
    "total_tokens_input": total_input_tokens,  
    # ...  
}
```

Variable Explanations:

- `self._requests`: List of all tracked LLM requests
- `r.input_tokens`: Number of input tokens for a single request (from Azure OpenAI via ADK)
- `total_input_tokens`: Accumulated sum of all input tokens across all requests

Data Source

Input token counts come from **Azure OpenAI's API response**, accessed through **Google ADK's `event.usage_metadata.prompt_token_count`**.

Flow:

1. Azure OpenAI counts tokens internally
2. Returns count in API response: `usage.prompt_tokens`
3. ADK wraps it: `event.usage_metadata.prompt_token_count`
4. Our code extracts: `input_tokens = usage.prompt_token_count`
5. Stored in: `LLMRequest.input_tokens`

3. Output Tokens Card

Definition

Displays the total number of output/completion tokens generated across all LLM requests.

Formula

Total Output Tokens = sum(output_tokens for all requests)

Code Snippet

Location: `backend/Agents/observability.py` (Line 297, 309)

```
# Sum all output tokens from all requests
total_output_tokens = sum(r.output_tokens for r in self._requests)

return {
    # ...
    "total_tokens_output": total_output_tokens,
    # ...
}
```

Variable Explanations:

- `self._requests`: List of all tracked LLM requests
- `r.output_tokens`: Number of output tokens for a single request (from Azure OpenAI via ADK)
- `total_output_tokens`: Accumulated sum of all output tokens across all requests

Data Source

Output token counts come from **Azure OpenAI's API response**, accessed through **Google ADK's `event.usage_metadata.candidates_token_count`**.

Flow:

1. Azure OpenAI counts tokens internally
2. Returns count in API response: `usage.completion_tokens`
3. ADK wraps it: `event.usage_metadata.candidates_token_count`
4. Our code extracts: `output_tokens = usage.candidates_token_count`
5. Stored in: `LLMRequest.output_tokens`

4. Total Tokens Card

Definition

Displays the combined total of input and output tokens across all requests.

Formula

$$\text{Total Tokens} = \text{Total Input Tokens} + \text{Total Output Tokens}$$

Code Snippet

Location: `backend/Agents/observability.py` (Lines 296-297, 310)

```
total_input_tokens = sum(r.input_tokens for r in self._requests)
total_output_tokens = sum(r.output_tokens for r in self._requests)

return {
    # ...
    "total_tokens": total_input_tokens + total_output_tokens,
    # ...
}
```

Variable Explanations:

- `total_input_tokens`: Sum of all input tokens
- `total_output_tokens`: Sum of all output tokens
- `total_input_tokens + total_output_tokens`: Combined total

Data Source

Calculated from the sum of input and output tokens (both from Azure OpenAI via ADK).

5. Avg Latency Card

Definition

Displays the average latency (response time) for backend processing of LLM requests, measured in milliseconds or seconds.

Formula

$$\text{Average Latency} = \text{sum}(\text{latency_ms for all requests}) / \text{count}(\text{all requests})$$

Code Snippet

Location: `backend/Agents/observability.py` (Line 301, 314)

```
# Calculate average latency across all requests
avg_latency = sum(r.latency_ms for r in self._requests) / len(self._requests)

return {
    # ...
    "average_latency_ms": avg_latency
}
```

Variable Explanations:

- `r.latency_ms`: Latency for a single request (time from backend receiving request to finishing processing)
- `sum(r.latency_ms for r in self._requests)`: Total latency across all requests
- `len(self._requests)`: Number of requests
- `avg_latency`: Average latency per request in milliseconds

Data Source

Latency is measured in `backend/api.py` using Python's `time.time()` function:

Measurement Flow:

```
# api.py - Line 66
start_time = time.time() # Start timer when backend receives request

# ... backend processing (ADK, agents, Azure OpenAI) ...
```

```
# api.py - Line 139
latency_ms = (time.time() - start_time) * 1000 # Stop timer, convert to
milliseconds
```

What's Measured:

-  Session management time
-  ADK orchestration time
-  Azure OpenAI API calls
-  Agent processing time
-  Source extraction time
-  NOT measured: Network time, UI rendering time

6. Input Cost Card

Definition

Displays the total cost in USD for all input/prompt tokens consumed.

Formula

```
Input Cost (USD) = sum((input_tokens / 1,000,000) × price_per_1M_input for all
requests)
```

Code Snippet

Step 1: Calculate cost per request

Location: `backend/Agents/observability.py` (Lines 218-220)

```
# Convert tokens to millions, multiply by price per million
input_cost = (input_tokens / TOKENS_PER_MILLION) * pricing_data["input_per_1m"]
# Example: (12,000 / 1,000,000) × $5.00 = 0.012 × $5.00 = $0.06
```

Step 2: Sum all input costs

Location: `backend/Agents/observability.py` (Line 299, 312)

```
# Sum input costs from all requests
input_cost = sum(r.input_cost for r in self._requests)

return {
    # ...
    "input_cost_usd": input_cost,
    # ...
}
```

Variable Explanations:

- `input_tokens`: Number of input tokens for this request
- `TOKENS_PER_MILLION`: Constant = 1,000,000 (for converting to millions)
- `pricing_data["input_per_1m"]`: Price per 1 million input tokens (e.g., \$5.00) - from LiteLLM or web
- `(input_tokens / TOKENS_PER_MILLION)`: Converts tokens to millions (e.g., 12,000 → 0.012 million)
- `input_cost`: Cost in USD for this request's input tokens
- `r.input_cost`: Cost for a single request
- `sum(r.input_cost for r in self._requests)`: Total input cost across all requests

Data Source - Pricing Strategies

The system uses **two strategies** to fetch pricing data, in order of preference:

Strategy 1: LiteLLM (Primary)

Location: `backend/Agents/observability.py` (Lines 163-198)

```
# LiteLLM maintains an up-to-date pricing database for all major LLM providers
# Source:
https://github.com/BerriAI/litellm/blob/main/model_prices_and_context_window.json
try:
    input_cost_per_token, output_cost_per_token = litellm.cost_per_token(
        model="gpt-4.1",
        prompt_tokens=input_tokens,
        completion_tokens=output_tokens
```

```

    )
    # LiteLLM returns the actual cost in USD (already calculated)
    input_cost = input_cost_per_token
    output_cost = output_cost_per_token
    return (input_cost + output_cost, input_cost, output_cost)
except Exception as e:
    # If LiteLLM doesn't have pricing for this model, fall back to Strategy 2
    print(f"LiteLLM pricing not available, trying internet search...")

```

Advantages:

-  Most reliable and up-to-date
-  Covers all major LLM providers
-  Updated when you upgrade LiteLLM package
-  Returns cost directly (no manual calculation needed)

Strategy 2: Internet Search (Fallback)

Location: `backend/Agents/observability.py` (Lines 205-223)

```

# If LiteLLM doesn't have pricing, scrape from langcopilot.com
pricing_data = self._fetch_pricing_from_web(model)

if pricing_data:
    # pricing_data format: {"input_per_1m": 5.00, "output_per_1m": 15.00}
    # Manual calculation required
    input_cost = (input_tokens / TOKENS_PER_MILLION) * pricing_data["input_per_1m"]
    output_cost = (output_tokens / TOKENS_PER_MILLION) *
pricing_data["output_per_1m"]
    return (input_cost + output_cost, input_cost, output_cost)

```

When Used:

- New models not yet in LiteLLM's database
- Custom models or fine-tuned models
- Models from newer providers

How It Works:

1. Converts model name to URL slug: "gpt-4/turbo" → "gpt-4-turbo"
2. Fetches pricing page: <https://www.langcopilot.com/llm-pricing/openai/gpt-4-turbo>
3. Scrapes HTML for pricing text: "\$5.00 per 1M input tokens"
4. Extracts numbers using regex

5. Returns: `{"input_per_1m": 5.00, "output_per_1m": 15.00, "source": "web"}`

If Both Strategies Fail:

```
# Raise an error with helpful instructions
raise ValueError(
    "No pricing data found for model 'xyz'. "
    "Please either:\n"
    "  1. Update LiteLLM package: pip install --upgrade litellm\n"
    "  2. Verify the model name is correct\n"
    "  3. Add manual pricing for this model"
)
```

Note: The pricing is always quoted as "\$ per 1 million tokens" (industry standard).

7. Output Cost Card

Definition

Displays the total cost in USD for all output/completion tokens generated.

Formula

Output Cost (USD) = $\text{sum}((\text{output_tokens} / 1,000,000) \times \text{price_per_1M_output for all requests})$

Code Snippet

Step 1: Calculate cost per request

Location: `backend/Agents/observability.py` (Line 220)

```
# Convert tokens to millions, multiply by price per million
output_cost = (output_tokens / TOKENS_PER_MILLION) * pricing_data["output_per_1m"]
# Example: (35,000 / 1,000,000) × $15.00 = 0.035 × $15.00 = $0.525
```

Step 2: Sum all output costs

Location: `backend/Agents/observability.py` (Line 300, 313)

```
# Sum output costs from all requests
output_cost = sum(r.output_cost for r in self._requests)

return {
    # ...
    "output_cost_usd": output_cost,
    # ...
}
```

Variable Explanations:

- `output_tokens`: Number of output tokens for this request
- `TOKENS_PER_MILLION`: Constant = 1,000,000 (for converting to millions)
- `pricing_data["output_per_1m"]`: Price per 1 million output tokens (e.g., \$15.00) - from LiteLLM or web
- `(output_tokens / TOKENS_PER_MILLION)`: Converts tokens to millions (e.g., 35,000 → 0.035 million)
- `output_cost`: Cost in USD for this request's output tokens
- `r.output_cost`: Cost for a single request
- `sum(r.output_cost for r in self._requests)`: Total output cost across all requests

Data Source

Uses the same **two-strategy pricing system** as Input Cost (see Input Cost section above):

1. **Strategy 1 (Primary)**: LiteLLM's built-in pricing database
2. **Strategy 2 (Fallback)**: Web scraping from langcopilot.com

Note: Output tokens are typically more expensive than input tokens because generating text requires more computation than processing it.

Example pricing:

- GPT-4: 5/1M *input* vs 15/1M output (3x more expensive)

- GPT-3.5: 0.50/1M *input* vs 1.50/1M output (3x more expensive)
-

8. Total Cost Card

Definition

Displays the total cost in USD for all LLM requests (input + output tokens combined).

Formula

```
Total Cost (USD) = Input Cost + Output Cost  
OR  
Total Cost (USD) = sum(cost for all requests)
```

Code Snippet

Location: `backend/Agents/observability.py` (Line 298, 311)

```
# Sum total costs from all requests  
total_cost = sum(r.cost for r in self._requests)  
  
return {  
    # ...  
    "total_cost_usd": total_cost,  
    # ...  
}
```

Variable Explanations:

- `r.cost`: Total cost for a single request (input_cost + output_cost)
- `sum(r.cost for r in self._requests)`: Accumulated total cost across all requests

How Cost is Calculated Per Request

Location: `backend/Agents/observability.py` (Line 253)

```
# When tracking a request
total_tokens = input_tokens + output_tokens
cost, input_cost, output_cost = self.calculate_cost(model, input_tokens,
output_tokens)

# Where:
# cost = input_cost + output_cost
# input_cost = (input_tokens / 1,000,000) × price_per_1M_input
# output_cost = (output_tokens / 1,000,000) × price_per_1M_output
```

Data Source

Calculated from the sum of all request costs, where each request's cost is the sum of its input and output costs.

Pricing obtained via two-strategy system:

1. **Strategy 1 (Primary):** LiteLLM's built-in pricing database
2. **Strategy 2 (Fallback):** Web scraping from langcopilot.com

See the [Input Cost section](#) for detailed explanation of how pricing is fetched.

Additional Dashboard Sections

Beyond the 8 main cards, the Observability Dashboard includes several detailed views for deeper analysis.

9. Overview Tab - System Health

Definition

Displays calculated metrics derived from the summary statistics, including success rate (with progress bar), average tokens per request, and cost per request.

Formulas

Success Rate = (successful_requests / total_requests) × 100
Average Tokens per Request = total_tokens / total_requests
Cost per Request = total_cost_usd / total_requests

Code Snippet

Location: `myf/src/pages/ObservabilityDashboard.tsx` (Lines 253-276)

```
// Success rate - from summary stats
{summary.success_rate.toFixed(1)}%

// Average tokens per request - calculated in frontend
{formatNumber(Math.round(summary.total_tokens / Math.max(summary.total_requests, 1)))}

// Cost per request - calculated in frontend
{formatCurrency(summary.total_cost_usd / Math.max(summary.total_requests, 1))}
```

Variable Explanations:

- `summary.success_rate`: Pre-calculated success rate from backend (percentage)
- `summary.total_tokens`: Total tokens across all requests (from backend)
- `summary.total_requests`: Total number of requests (from backend)
- `summary.total_cost_usd`: Total cost in USD (from backend)
- `Math.max(summary.total_requests, 1)`: Prevents division by zero

Data Source

All data comes from `get_summary_stats()` which aggregates the `self._requests` list. These are derived metrics calculated from the summary data.

10. By Model Tab

Definition

Groups all requests by model name and displays aggregated statistics for each model, including request count, token usage, costs, average latency, and success rate.

Formulas

Per Model:

- requests = count(requests where model == this_model)
- tokens_input = sum(input_tokens where model == this_model)
- tokens_output = sum(output_tokens where model == this_model)
- tokens_total = tokens_input + tokens_output
- cost_usd = sum(cost where model == this_model)
- average_latency_ms = sum(latency_ms) / count(requests) for this model
- success_rate = (successful_requests / total_requests) × 100 for this model

Code Snippet

Location: `backend/Agents/observability.py` (Lines 332-358)

```
# Group requests by model
for request in self._requests:
    stats = model_stats[request.model]
    stats["requests"] += 1
    stats["tokens_input"] += request.input_tokens
    stats["tokens_output"] += request.output_tokens
    stats["tokens_total"] += request.total_tokens
    stats["cost_usd"] += request.cost
    stats["input_cost_usd"] += request.input_cost
    stats["output_cost_usd"] += request.output_cost
    stats["latency_total"] += request.latency_ms
    if request.success:
        stats["successful"] += 1

# Calculate averages per model
result[model] = {
    "requests": stats["requests"],
    "tokens_input": stats["tokens_input"],
    "tokens_output": stats["tokens_output"],
    "tokens_total": stats["tokens_total"],
    "cost_usd": stats["cost_usd"],
    "average_latency_ms": stats["latency_total"] / stats["requests"] if
stats["requests"] > 0 else 0,
    "success_rate": (stats["successful"] / stats["requests"]) * 100 if
stats["requests"] > 0 else 0
}
```

Variable Explanations:

- `self._requests`: List of all tracked requests
- `request.model`: Model name for a single request (e.g., "gpt-4.1")
- `model_stats`: Dictionary using `defaultdict` to group stats by model name
- `stats["requests"]`: Counter for requests using this model
- `stats["tokens_input"]`: Accumulated input tokens for this model
- `stats["latency_total"]`: Accumulated latency for averaging
- `stats["successful"]`: Counter for successful requests for this model

Data Source

Iterates through `self._requests` and groups all metrics by the `request.model` field. Each model gets its own aggregated statistics.

11. Hourly Breakdown Tab

Definition

Displays request counts and total costs grouped by hour for the last 24 hours (configurable), with the most recent hour displayed first.

Formulas

```
For each hour in last 24 hours:
- hour_start = current_time - timedelta(hours=i+1)
- hour_end = current_time - timedelta(hours=i)
- requests_in_hour = count(requests where hour_start <= timestamp < hour_end)
- cost_in_hour = sum(cost for requests where hour_start <= timestamp < hour_end)
```

Code Snippet

Location: `backend/Agents/observability.py` (Lines 376-391)

```

# Loop through each hour (default: 24 hours)
for i in range(hours):
    hour_start = now - timedelta(hours=i+1)
    hour_end = now - timedelta(hours=i)

    # Filter requests within this hour
    hour_requests = [
        r for r in self._requests
        if hour_start <= datetime.fromisoformat(r.timestamp) < hour_end
    ]

    # Calculate stats for this hour
    hourly_data.append({
        "hour": hour_start.strftime("%Y-%m-%d %H:00"),
        "requests": len(hour_requests),
        "cost": sum(r.cost for r in hour_requests)
    })

return list(reversed(hourly_data)) # Most recent first

```

Variable Explanations:

- **hours**: Number of hours to include (default: `DEFAULT_HOURLY_BREAKDOWN_HOURS` = 24)
- **now**: Current datetime
- **hour_start**: Start of this hour's time window
- **hour_end**: End of this hour's time window
- **hour_requests**: Filtered list of requests within this hour's window
- **r.timestamp**: ISO format timestamp string for each request
- **datetime.fromisoformat()**: Converts ISO string to datetime for comparison
- **list(reversed())**: Reverses list to show most recent hour first

Data Source

Filters `self._requests` by timestamp, grouping requests into hourly buckets. Uses Python's `datetime` and `timedelta` for time-based filtering.

12. Recent Requests Tab

Definition

Displays a detailed table of the most recent LLM requests (default: last 50), sorted by timestamp with the newest first, showing timestamp, model, tokens, cost, latency, and success status.

Formula

```
Recent Requests = sorted(all_requests, by=timestamp, descending=True)[:limit]
```

Code Snippet

Location: `backend/Agents/observability.py` (Lines 404-405)

```
# Sort all requests by timestamp (newest first), take top N
recent = sorted(self._requests, key=lambda r: r.timestamp, reverse=True)[:limit]

# Convert LLMRequest objects to dictionaries
return [asdict(request) for request in recent]
```

Variable Explanations:

- `self._requests`: List of all tracked LLM requests
- `limit`: Maximum number of requests to return (default: `DEFAULT_RECENT_REQUESTS_LIMIT = 50`)
- `key=lambda r: r.timestamp`: Sort key - uses the timestamp field
- `reverse=True`: Sort descending (newest first)
- `[:limit]`: Slice to get only the first N items
- `asdict(request)`: Converts dataclass `LLMRequest` to dictionary for JSON serialization

Data Source

Sorts the entire `self._requests` list by timestamp and returns the most recent entries. The `asdict()` function from Python's `dataclasses` module converts each `LLMRequest` object into a dictionary containing all fields (timestamp, model, input_tokens, output_tokens, cost, latency_ms, success, error, etc.).

Frontend Display: The frontend receives this data and displays each request's:

- Timestamp (formatted as locale string)
 - Model name
 - Tokens (total with input/output breakdown)
 - Cost in USD
 - Latency in ms/seconds
 - Success status (badge with icon)
-

Last Updated: 2026-01-19